

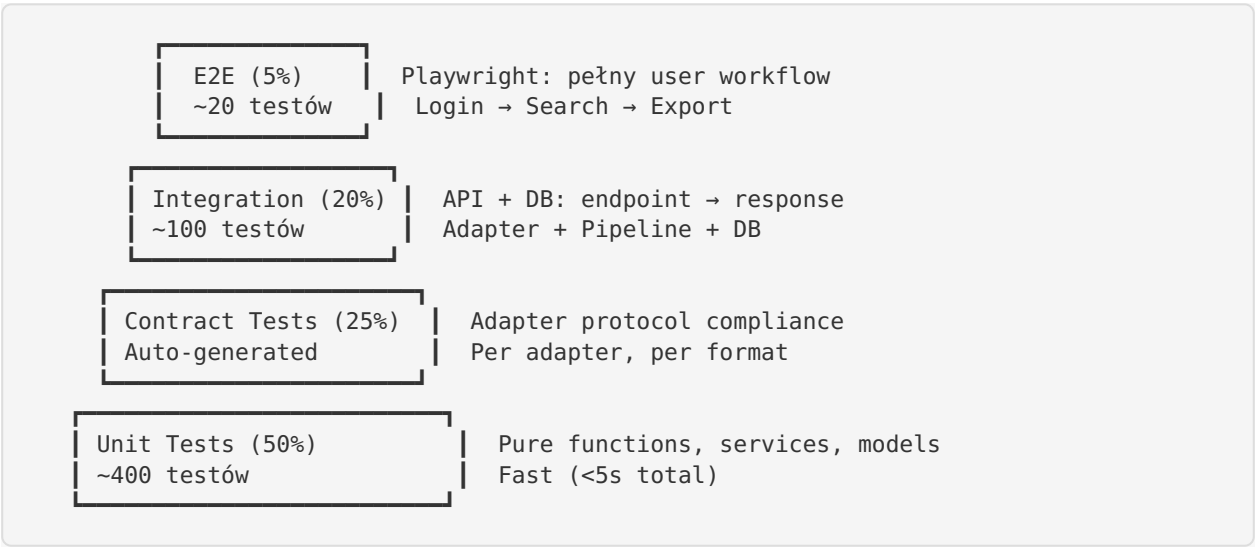
07 — QA Engineer

[← Powrót do README](#) (../README.md) | [← Security Engineer](#) (./security-engineer.md) | [Następna: Product Owner](#) → (./product-owner.md)

Zakres odpowiedzialności

- Strategia testów (test pyramid, test plan per milestone)
- Test automation framework (pytest, Playwright)
- Performance testing (load, stress, spike)
- Security testing support (OWASP ZAP)
- Regression testing i quality gates
- Test data management
- Bug triage i reporting

Test Strategy — Test Pyramid



Test Automation Framework

Narzędzia

Narzędzie	Typ	Use case
pytest	Unit + Integration	Główny runner, fixtures, parametrize
pytest-cov	Coverage	Line + branch coverage
responses	HTTP mocking	Mock external API calls
fakeredis	Redis mocking	Cache/session tests
Playwright	E2E	Browser-based user flows
Locust	Performance	Load/stress testing
OWASP ZAP	Security	Automated security scan
testcontainers	Integration	Real PostgreSQL in tests

Fixture Strategy

```
# tests/conftest.py

@pytest.fixture(scope="session")
def pg_container():
    """Real PostgreSQL container for integration tests."""
    with PostgresContainer("postgres:16") as pg:
        yield pg

@pytest.fixture
def db_session(pg_container):
    """Transaction-scoped DB session (auto-rollback)."""
    engine = create_engine(pg_container.get_connection_url())
    Base.metadata.create_all(engine)
    session = Session(engine)
    yield session
    session.rollback()
    session.close()

@pytest.fixture
def sample_indicators(db_session):
    """Factory: generuj N przykładowych indicatorów."""
    def _create(count=10, source="test", ioc_type="ip"):
        indicators = []
        for i in range(count):
            ind = Indicator(
                value=f"10.0.0.{i}",
                type=ioc_type,
                source=source,
                source_id=f"test_{i}",
                confidence=80,
                is_active=True,
            )
            db_session.add(ind)
            indicators.append(ind)
        db_session.flush()
        return indicators
    return _create

@pytest.fixture
def authenticated_client(app, db_session):
    """Flask test client z zalogowanym admin user."""
    from app.models import User
    user = User(username="testadmin", role="admin")
    user.set_password("TestPassword123!")
    db_session.add(user)
    db_session.commit()

    client = app.test_client()
    client.post("/auth/login", data={
        "username": "testadmin",
        "password": "TestPassword123!",
    })
    return client
```

Performance Testing

Locust Configuration

```
# tests/performance/locustfile.py
from locust import HttpUser, task, between

class IOCSERVICEUser(HttpUser):
    wait_time = between(1, 3)

    @task(5)
    def search_indicators(self):
        self.client.get("/api/v1/indicators?query=ip:10.0.*&limit=50")

    @task(3)
    def export_csv(self):
        self.client.get("/api/v1/indicators/export?format=csv&limit=1000")

    @task(1)
    def health_check(self):
        self.client.get("/api/v1/health")

    @task(1)
    def feed_status(self):
        self.client.get("/api/v1/feeds")
```

Performance Targets

Metryka	Target	Typ testu
Response time p50	<100ms	Load test
Response time p95	<200ms	Load test
Response time p99	<500ms	Load test
Throughput	>1000 req/s	Load test
Error rate	<0.1%	Load test
Max concurrent users	100	Stress test
Export 100k IOC	<10s	Benchmark
Feed sync (10k IOC)	<30s	Benchmark

Quality Gates dla CI/CD

Gate	Threshold	Block Merge?
Unit test coverage	$\geq 80\%$	✓ Yes
Branch coverage	$\geq 70\%$	✓ Yes
0 critical bugs	0	✓ Yes
0 high bugs	0	✓ Yes
Security scan clean	0 high/critical	✓ Yes
Performance regression	<10% degradation	⚠ Warning
Linting	0 errors	✓ Yes
Type checking (mypy)	0 errors	⚠ Warning

Bug Triage — Priority Matrix

	High Impact	Low Impact
High Probability	🔴 P1 — Fix immediately	🟡 P2 — Fix in current sprint
Low Probability	🟡 P2 — Fix in current sprint	🟢 P3 — Backlog

Severity Definitions

Severity	Definicja	Przykład
S1 Critical	System down, data loss	DB crash, auth bypass
S2 High	Feature broken, no work-around	Export fails, feed not syncing
S3 Medium	Feature broken, workaround exists	Search filter not working
S4 Low	Cosmetic, minor inconvenience	Typo, alignment issue

Test Reporting & Metrics

Dashboard Metryki

- **Test pass rate** — target: >99% per sprint
 - **Coverage trend** — target: $\geq 80\%$, never decreasing
 - **Bug escape rate** — bugs found in prod / total bugs
 - **Defect density** — bugs per 1000 LOC
 - **Mean time to detect** — czas od wprowadzenia bugu do wykrycia
 - **Test execution time** — target: <5 min (unit), <15 min (full)
-

[← Security Engineer](#) (./security-engineer.md) | [Następna: Product Owner](#) → (./product-owner.md)